

Coding Problem 1: The "HealthSync" Patient Record System

Scenario Background:

You are the lead backend developer for *HealthSync*, a modern healthcare platform. You are tasked with designing the core data model and persistence layer for tracking patient medical histories and their associated billing accounts.

The system needs to manage different types of medical records (e.g., Prescriptions and Lab Results) under a unified system. Additionally, the finance department needs a highly optimized, read-only report generated daily that aggregates patient billing data.

Implementation Constraints & Requirements:

1. Entity Mapping & Inheritance Constraints:

- **The Inheritance Strategy:** You must implement a `MedicalRecord` base entity. From this, you must derive two concrete entities: `PrescriptionRecord` and `LabResultRecord`.
 - *Constraint:* You **MUST** use the `SINGLE_TABLE` inheritance strategy. You must explicitly define the Discriminator Column (named `record_type`) and the specific discriminator values for each child class.
- **The Bidirectional Relationship:**
 - Create a `Patient` entity.
 - A `Patient` has a One-To-Many relationship with `MedicalRecord`.
 - *Constraint:* The relationship **MUST** be bidirectional. You must explicitly define the "Owning Side" and the "Inverse Side" using the correct annotations (`@JoinColumn` and `mappedBy`).
 - *Constraint:* If a `Patient` is deleted from the database, all of their associated `MedicalRecord` entries **MUST** be automatically deleted by the JPA provider.
- **The One-To-One Mapping:**
 - Create a `BillingAccount` entity.
 - A `Patient` has a One-To-One relationship with a `BillingAccount`.
 - *Constraint:* The `BillingAccount` must share the exact same Primary Key as the `Patient` (i.e., the `patient_id` serves as both the PK and FK in the billing table). You must use `@MapsId` to achieve this.

2. JPQL Constraints:

- The finance department requires a specific Data Transfer Object (DTO) called `PatientBillingSummaryDTO` containing: `String patientName`, `String recordType`, and `BigDecimal currentBalance`.
- *Constraint:* Write a single JPQL query that joins the `Patient`, `MedicalRecord`, and `BillingAccount` entities.
- *Constraint:* The JPQL query **MUST NOT** return raw Entity objects. It must directly instantiate and return instances of `PatientBillingSummaryDTO` using a **JPQL Constructor Expression**.

3. Repository Constraints:

- Create a Spring Data JPA interface called PatientRepository.
- *Constraint:* You must define the JPQL query from Step 2 inside this interface using the @Query annotation.
- *Constraint:* You must write a Spring Data "Derived Query Method" (no @Query annotation allowed) that finds all Patients whose BillingAccount balance is strictly greater than a provided BigDecimal amount and who have at least one PrescriptionRecord containing a specific medication name.

Why this makes a great interview/study question:

1. **SINGLE_TABLE Inheritance Mastery:** This tests if the candidate knows how to physically map OOP inheritance to a relational database. The SINGLE_TABLE strategy is highly performant but requires an understanding of @DiscriminatorColumn.
2. **The @MapsId Optimization:** Many developers default to a standard foreign key for One-To-One relationships. By forcing @MapsId, it tests the candidate's knowledge of advanced database normalization. Sharing the PK eliminates the need for an extra unique index and guarantees strict 1:1 cardinality at the database level.
3. **JPQL Constructor Expressions:** Returning massive managed Entity graphs just to display a simple report is a major performance killer (wasting memory and dirty-checking CPU cycles). Testing constructor expressions (SELECT new com.package.DTO(...)) proves the candidate knows how to write high-performance, read-only projection queries.
4. **Spring Data Property Traversal:** The derived query method tests the candidate's ability to navigate deep object graphs (from Patient down into BillingAccount and down into the polymorphic records collection) using solely Spring Data's naming conventions.

Coding Problem 2: The "CartFlux" Order Fulfillment Engine

Scenario Background:

You are the lead backend developer for CartFlux, an enterprise e-commerce platform. The system must manage PurchaseOrder entities and Product entities.

Because an order can contain many products, and a product can be in many orders, this is fundamentally a Many-To-Many relationship. However, the warehouse team needs to know the exact *quantity* and *unit price at the time of purchase* for each product in an order. Standard @ManyToMany annotations cannot store these extra fields in the join table.

Furthermore, the fulfillment dashboard suffers from the dreaded "N+1 Select Problem," querying the database hundreds of times just to load a single list of pending orders.

Implementation Constraints & Requirements:

1. Entity Mapping Constraints:

- **The M:N with Payload Strategy:** You must break the Many-To-Many relationship into two One-To-Many relationships using a middle entity called OrderLineItem.
- **Constraint:** You must use a Composite Primary Key for OrderLineItem using the @Embeddable and @EmbeddedId annotations. The composite key must consist of the order_id and product_id.
- **Constraint:** The OrderLineItem must store int quantity and BigDecimal lockedPrice.

2. JPQL Constraints:

- The fulfillment dashboard needs to load all "PENDING" orders along with their complete list of line items and associated products.
- **Constraint:** Write a single JPQL query that fetches the PurchaseOrder, its OrderLineItems, and the Products in exactly ONE database trip to eliminate the N+1 problem. You MUST use the JOIN FETCH keyword.

3. Repository Constraints:

- Create a PurchaseOrderRepository interface.
- **Constraint:** Define the JPQL query from Step 2 using @Query.
- **Constraint:** Write a Spring Data "Derived Query Method" that finds all PurchaseOrders placed between two LocalDateTime timestamps where the customerEmail ends with a specific domain (e.g., "@gmail.com").

Why this makes a great interview/study question:

- **The Many-To-Many Payload Problem:** This is a classic enterprise hurdle. A standard @ManyToMany creates a hidden join table, which makes it impossible to store quantities. Forcing the use of @EmbeddedId and @MapsId proves the developer understands deep JPA key mapping.
- **N+1 Selects & JOIN FETCH:** The N+1 problem is the #1 cause of ORM performance bottlenecks. By mandating JOIN FETCH, you test if the candidate knows how to override FetchType.LAZY dynamically within a query to pull massive object graphs efficiently.

Coding Problem 3: The "CorpMatrix" Employee Hierarchy

Scenario Background:

You are developing an HR management system called CorpMatrix. The system tracks employees, dividing them into FullTimeEmployees and Contractors.

Furthermore, the company structure is heavily nested. Every employee can report to a manager, and that manager is *also* an employee in the same table. The HR department needs an organizational report that groups employees by their manager and counts how many direct reports each manager has.

Implementation Constraints & Requirements:

1. Entity Mapping & Inheritance Constraints:

- **The Inheritance Strategy:** Create an abstract Employee base class. Implement JOINED inheritance for the FullTimeEmployee and Contractor subclasses.
- **The Self-Referencing Relationship:**
 - **Constraint:** Inside the Employee base class, create a @ManyToOne relationship to represent the manager.
 - **Constraint:** Create a @OneToMany collection representing the directReports. Set up the mappedBy relationship correctly so they point to each other in the same table.

2. JPQL Constraints:

- HR needs a DTO called ManagerSpanDTO containing: String managerName and Long directReportCount.
- **Constraint:** Write a single JPQL query that returns a list of ManagerSpanDTOs. You must use an INNER JOIN traversing the self-referencing relationship, and use the GROUP BY and COUNT() aggregate functions.

3. Repository Constraints:

- Create an EmployeeRepository.
- **Constraint:** Define the DTO aggregation query using @Query.
- **Constraint:** Write a derived query method that finds all Employees whose manager has a specific firstName and whose department matches a provided string.

Why this makes a great interview/study question:

- **JOINED Inheritance:** Tests understanding of polymorphic database design. Unlike SINGLE_TABLE, JOINED normalizes the data perfectly (no null columns for contractors), at the cost of requiring SQL JOINS when querying the base class.
- **Self-Referencing Entities:** Representing hierarchical data (trees/graphs) inside a relational database is conceptually difficult. Proving they can map manager to directReports within the exact same entity class is a hallmark of a strong backend developer.

Coding Problem 4: The "StreamCast" Content Delivery Network

Scenario Background:

You are the architect for StreamCast, a global media platform. The system hosts a diverse array of media. You have an Author entity that produces MediaAssets.

MediaAsset is an abstract base class extended by VideoAsset and PodcastAsset. Because the metadata for Videos and Podcasts are completely different and never queried together, the database team has requested you use the TABLE_PER_CLASS strategy.

The front-end needs to find all media by a specific author, but they specifically want to filter the query to *only* return VideoAssets that are in '4K' resolution.

Implementation Constraints & Requirements:

1. Entity Mapping Constraints:

- **The Inheritance Strategy:** Create the Author, MediaAsset, VideoAsset, and PodcastAsset entities.
- **Constraint:** You MUST use the TABLE_PER_CLASS strategy.
- **Constraint:** Create a unidirectional @OneToMany relationship from Author to MediaAsset.

2. JPQL Constraints (Polymorphic Downcasting):

- **Constraint:** Write a JPQL query that selects MediaAsset objects associated with a specific Author ID.
- **Constraint:** You MUST use the JPQL TREAT operator in the WHERE clause to explicitly downcast the MediaAsset to a VideoAsset so you can filter by the child-specific resolution property.

3. Repository Constraints:

- Create an AuthorRepository.
- **Constraint:** When fetching an Author by their username, use a Spring Data @EntityGraph annotation to eagerly fetch their associated MediaAsset list, bypassing the default lazy loading behavior without writing a custom JPQL JOIN FETCH.

Why this makes a great interview/study question:

- **The TREAT Operator:** Most developers only query base classes or specific child classes. Querying a base class but filtering on a child class's specific property is impossible without TREAT, making it a rare but incredibly powerful tool for polymorphic datasets.
- **@EntityGraph vs JOIN FETCH:** It proves the candidate is up-to-date with modern Spring Data JPA. EntityGraphs allow developers to maintain purely generated derived queries (findByUsername) while still solving the N+1 eager-fetching problem cleanly via annotations.

Coding Problem 5: The "AeroFleet" Aircraft Maintenance Log

Scenario Background:

You are building the software for AeroFleet, a commercial airline. The system tracks Aircraft entities.

Over its lifetime, an aircraft undergoes hundreds of minor maintenance checks. These logs do not have their own primary keys and do not need to exist independently of the Aircraft. The database team insists these should just be stored as a tightly-coupled collection in a secondary table.

The maintenance dashboard requires a paginated list of aircraft that have undergone a specific type of maintenance action within a specific time frame.

Implementation Constraints & Requirements:

1. Entity Mapping Constraints:

- **The Collection Strategy:** You must NOT create a `@OneToMany` relationship with a separate `@Entity`.
- **Constraint:** You must use the `@ElementCollection` and `@CollectionTable` annotations to map a list of `@Embeddable MaintenanceLog` objects directly to the Aircraft entity.

2. JPQL Constraints:

- **Constraint:** Write a JPQL query that searches the Aircraft entities based on the properties inside the embedded `MaintenanceLog` collection.
- **Constraint:** The query must support Spring Data pagination to ensure massive result sets don't crash the server.

3. Repository Constraints:

- Create an `AircraftRepository`.
- **Constraint:** Define the paginated JPQL query from Step 2, ensuring it returns a `Page<Aircraft>` and accepts a `Pageable` parameter.
- **Constraint:** Write a derived query method that finds all Aircrafts where the `modelName` is inside a provided `List<String>` and the `isGrounded` boolean is true.

Why this makes a great interview/study question:

- **@ElementCollection vs @OneToMany:** Junior developers turn *everything* into an Entity with an ID. Teaching developers to identify "Value Objects" (data that has no identity of its own) and mapping them cleanly using `@ElementCollection` drastically simplifies persistence layers and avoids unnecessary repository clutter.
- **Pagination at the DB level:** Passing `Pageable` into a `@Query` proves the candidate understands how to let the database handle LIMIT and OFFSET natively, rather than returning a massive List and slicing it in Java memory, which guarantees a scalable architecture.

Problem 1: The Stale Prototype Trap (Scope Resolution Defect)

Enterprise Scenario: A highly concurrent PDF report generation service relies on a ReportManager (Singleton) to orchestrate tasks. To ensure isolated data processing, the ReportManager injects a ReportTask bean, which is strictly annotated with `@Scope("prototype")`. However, production logs show concurrent users are receiving each other's data because the same ReportTask instance is being reused.

Task: Refactor the ReportManager so it successfully requests a fresh, isolated ReportTask instance for every execution thread without using the new keyword or tightly coupling the code to the ApplicationContext.

Code Breakdown & Container Analysis:

- **The Defect:** If you inject ReportTask directly into ReportManager's constructor, Spring does this injection *once* during application startup. The prototype becomes trapped inside the singleton, rendering the `@Scope("prototype")` useless.
- **The Fix (ObjectFactory<T>):** By injecting an ObjectFactory, we defer the bean lookup. When `taskFactory.getObject()` is called inside the method, it reaches back into the Spring IoC container and triggers the prototype creation logic *at runtime*, returning a perfectly isolated instance for that specific thread.

Problem 2: The Circular Deadlock (Injection Architecture Breakdown)

Enterprise Scenario: A team transitioned all `@Autowired` field injections to strict Constructor Injection. Following the commit, the Spring Boot application crashes immediately on startup, throwing a fatal `BeanCurrentlyInCreationException`.

Task: Map out the exact failure and architect a resolution by refactoring the dependencies to break the deadlock using Setter Injection, explaining how Spring utilizes two-phase initialization.

Code Breakdown & Container Analysis:

- **The Defect:** If both classes use Constructor Injection, Spring attempts to instantiate OrderService but sees it needs InventoryService. It attempts to instantiate InventoryService but sees it needs OrderService. It enters an infinite loop, ultimately throwing `BeanCurrentlyInCreationException`.
 - **The Fix (Two-Phase Initialization):** Constructor injection forces Spring to provide a fully initialized bean immediately. By changing OrderService to use **Setter Injection**, we allow Spring to execute Phase 1 (Instantiation): It creates OrderService via the empty constructor and puts it in the container. Then it creates InventoryService (injecting the now-available OrderService). Finally, Phase 2 (Population) occurs: Spring calls `setInventoryService()` to finish wiring OrderService.
-

Problem 3: The Stateful Singleton Race Condition (Concurrency Failure)

Enterprise Scenario: A high-throughput `PaymentProcessor` annotated with `@Service` calculates transaction fees. A junior engineer extracted `transactionId` and `calculatedFee` into private class-level variables to "clean up" method signatures. Randomly, users are being charged fees belonging to completely different transactions.

Task: Refactor the `PaymentProcessor` to achieve strict thread-safe, enterprise-grade engineering quality by passing state via the Thread Stack rather than the Singleton Heap.

Code Breakdown & Container Analysis:

- **The Defect:** Spring creates only *one* instance of `@Service` classes (Singleton), which lives in the Shared Heap memory. If 1,000 HTTP requests hit the controller simultaneously, 1,000 different threads enter this single object. If they write to a class-level `transactionId` variable, they overwrite each other's data (Race Condition).
- **The Fix:** Move the variables inside the methods! Method parameters and local variables live in the **Thread Stack**, not the Heap. Even if 1,000 threads hit the `processPayment` method at the exact same microsecond, the JVM creates 1,000 isolated Stack Frames, guaranteeing absolute thread safety.

Problem 4: Ambiguous Infrastructure Contracts (`NoUniqueBeanDefinitionException`)

Enterprise Scenario: A notification system relies on a `NotificationService` interface. The container has three implementations: Email, Sms, and Push. The IoC container throws a `NoUniqueBeanDefinitionException`.

Task: Resolve this ambiguity using two distinct enterprise patterns: `@Qualifier` for a targeted fix, and `List<T>` injection for a dynamic broadcast fix.

Code Breakdown & Container Analysis:

- **`@Qualifier("sms")`:** The container looks at the interface, sees 3 implementations, and uses the string identifier to narrow the selection down to 1, fulfilling the strict constructor contract without violating fail-fast principles.
- **`List<NotificationService>`:** This is a highly scalable Enterprise Strategy Pattern. Spring natively understands Collection injection. It gathers all matching beans into a List. If a developer adds a `SlackNotificationService` tomorrow, they just annotate it with `@Service`—the `SystemAlertService` will automatically pick it up and execute it without modifying any existing code (Open/Closed Principle).

Problem 5: The Lifecycle Interceptor (BeanPostProcessor Manipulation)

Enterprise Scenario: A fintech compliance team dictates that every bean implementing the Auditable interface must be logged to a central SecurityRegistry exactly *after* Spring injects its dependencies, but *before* it processes traffic.

Task: Architect a custom BeanPostProcessor to intercept beans dynamically and execute the registration logic, demonstrating absolute control over the IoC container.

Code Breakdown & Container Analysis:

- **BeanPostProcessor:** This is a powerful Spring framework hook. When registered as a `@Component`, Spring passes *every single bean it creates* through this class.
- **instanceof Auditable:** Because hundreds of beans flow through here (including internal Spring beans), we use `instanceof` to dynamically filter only the beans conforming to our specific contract.
- **The Return Statement:** The returned object is what gets placed into the final IoC container. If you return null here, the bean is erased from existence. Advanced developers use this phase to wrap original beans in AOP (Aspect-Oriented Programming) Proxies before returning them.